

Disciplina: Aprendizado Profundo

Aula 7: Redes Neurais Convolucionais (CNNs)

Eliezer de Souza da Silva
sereliezer.github.io
eliezer.silva@ufc.br

Mestrado e Doutorado em Ciência da Computação, Universidade Federal do
Ceará (MDCC / UFC)

6 de Outubro de 2025

Roteiro da Aula de Hoje

- 1 Revisão e Motivação
- 2 O Princípio da Invariância e Simetrias
- 3 A Operação de Convolução
- 4 A Arquitetura de uma CNN
- 5 Laboratório e Próximos Passos

O Problema com MLPs para Imagens I

As Limitações dos MLPs

Embora os MLPs sejam aproximadores universais, eles não são ideais para dados com estrutura espacial, como imagens.

- **Perda da Estrutura Espacial:** Para usar um MLP, precisamos “achatar” a imagem (e.g., $28 \times 28 \rightarrow 784$), perdendo a informação de que pixels vizinhos estão relacionados.
- **Explosão de Parâmetros:** Uma única camada oculta com 500 neurônios para uma imagem de 28×28 já requer $784 \times 500 + 500 \approx 400.000$ parâmetros. Isso é computacionalmente caro e propenso a overfitting.

O Problema com MLPs para Imagens II

- **Falta de Invariância à Translação:** Um MLP treinado para reconhecer um objeto no canto superior esquerdo não saberá reconhecer o mesmo objeto no canto inferior direito. Ele precisa reaprender o padrão para cada nova posição.

A Inspiração: O Córtex Visual I

Hubel e Wiesel (1959)

Os neurocientistas David Hubel e Torsten Wiesel descobriram que o córtex visual dos mamíferos possui uma organização hierárquica.

- **Neurônios Simples:** Detectam características básicas em pequenas regiões do campo visual (campos receptivos), como bordas com orientações específicas.
- **Neurônios Complexos:** Recebem sinais de múltiplos neurônios simples, respondendo a padrões mais complexos de forma mais invariante à sua posição exata.

A Inspiração: O Córtex Visual II

Princípios para a Visão Computacional

Esta pesquisa inspirou três ideias fundamentais para as CNNs:

- 1 A visão começa com a detecção de **features locais e simples**.
- 2 O sistema visual constrói uma **representação hierárquica** do mundo.
- 3 A arquitetura deve ter algum grau de **invariância à translação**.

Codificando Conhecimento Prévio: Viés Indutivo I

Invariância à Translação

A principal **simetria** presente em imagens é a *invariância à translação*: a identidade de um objeto não muda se ele for deslocado. Um gato continua sendo um gato, não importa onde ele esteja na imagem.

Codificando Conhecimento Prévio: Viés Indutivo II

O Viés Indutivo das CNNs

As Redes Neurais Convolucionais (CNNs) são projetadas com um **viés indutivo** que explora essa simetria. Em vez de aprender tudo do zero, a arquitetura da CNN impõe três suposições importantes sobre o mundo visual:

- 1 Localidade:** As interações entre pixels são locais. Um pixel é mais influenciado por seus vizinhos.
- 2 Compartilhamento de Pesos:** As regras para detectar um padrão são as mesmas em toda a imagem. Um único “detector de features” pode ser deslizado sobre a imagem.
- 3 Hierarquia:** Features complexas são compostas por features mais simples, que por sua vez são compostas por bordas e cantos.

Além do Viés Indutivo: Derivando a Convolução I

Uma Pergunta Mais Profunda

Já vimos que as CNNs usam um viés indutivo (localidade, compartilhamento de pesos) que funciona bem para imagens. Mas podemos **derivar** a estrutura convolucional a partir de um princípio mais fundamental?

Além do Viés Indutivo: Derivando a Convolução II

O Princípio da Redução da Taxa de Invariância

Vamos revisitar nosso objetivo de aprendizado. Queremos encontrar uma representação z dos dados x que seja:

- 1 Compressiva:** As representações para dados da mesma classe devem ser compactas (baixa taxa).
- 2 Discriminativa:** As representações de classes diferentes devem ser “incoerentes” ou separáveis (alta taxa).

Agora, adicionamos uma terceira condição crucial: a representação deve ser **invariante** a um grupo de transformações $\mathcal{G}$ (e.g., translações, rotações).

Invariância de Grupo e a Estrutura Convolutacional I

Invariância vs. Equivariância

- **Invariância de Grupo:** A *identidade* do objeto (e.g., o rótulo da classe) não muda sob uma transformação. $f(\mathcal{T}(x)) = f(x)$.
- **Equivariância de Grupo:** Se a entrada é transformada, a *representação* (mapa de features) é transformada da mesma maneira. $\phi(\mathcal{T}(x)) = \mathcal{T}(\phi(x))$.

Uma CNN busca aprender features **equivariantes** que, após uma etapa final (como pooling global), produzem uma predição **invariante**.

Invariância de Grupo e a Estrutura Convolutiva II

Da Invariância à Convolução

Como podemos forçar a invariância no nosso modelo? A resposta de Buchanan et al. (2025) é elegante: consideramos todas as versões transladas de uma amostra x como pertencentes à mesma classe.

Se fizermos isso para dados 1D com translações circulares, a matriz de covariância dos dados “aumentados” se torna uma **matriz circulante**.

Uma propriedade matemática fundamental é que qualquer operador linear (como as matrizes de projeção no nosso objetivo de aprendizado) aplicado a dados com essa estrutura **também se torna uma matriz circulante**.

E a multiplicação por uma matriz circulante é, por definição, uma operação de convolução!

O Trade-off Fundamental: Invariância vs. Discriminabilidade

O Problema da Invariância Total

Se forcarmos a invariância a todas as translações possíveis para cada amostra, a representação de *todas as classes* pode colapsar para o mesmo subespaço, tornando a classificação impossível. A invariância total destrói a informação discriminativa.

O Trade-off Fundamental: Invariância vs. Discriminabilidade II

A Solução: Esparsidade e Múltiplos Canais

A saída para este impasse é assumir uma estrutura adicional nos dados: a **esparsidade**.

- A ideia é que os sinais de cada classe não são arbitrários, mas sim combinações *esparsas* de um “dicionário” de padrões ou “motivos” básicos.
- Para capturar esses padrões, primeiro “levantamos” (lift) o sinal para um espaço de dimensão maior usando múltiplos filtros (ou canais), gerando respostas esparsas.
- A rede neural profunda então opera sobre essas representações esparsas e multi-canal para encontrar subespaços que são ao mesmo tempo **invariantes à translação** e **linearmente separáveis**.

Definição Formal: Convolução Contínua 1D I

A Perspectiva do Processamento de Sinais

No domínio contínuo, a convolução de uma função (sinal) $x(t)$ com outra (o kernel) $k(t)$ é definida pela integral:

$$(x * k)(t) = \int_{-\infty}^{\infty} x(\tau)k(t - \tau)d\tau$$

Intuição: A saída em um tempo t é uma média ponderada de todo o sinal de entrada, onde a ponderação é dada pelo kernel "invertido" e deslizado, $k(t - \tau)$.

Convolução Discreta 1D I

A Versão para Dados Discretos

Para sinais discretos (como séries temporais ou os pixels de uma imagem), a integral se torna uma soma. A convolução discreta de um sinal x com um kernel k é:

$$(x * k)[n] = \sum_{m=-\infty}^{\infty} x[m]k[n - m]$$

Em aprendizado profundo, usamos kernels de tamanho finito e a **correlação cruzada**, que é computacionalmente equivalente (sem a inversão do kernel) e mais fácil de implementar.

Convolução Discreta 1D II

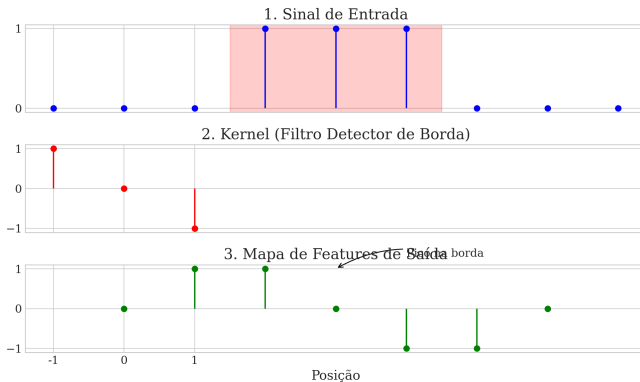


Figura: Um kernel $[1, 0, -1]$ desliza sobre o sinal. A saída é alta onde o padrão (uma borda) é detectado.

Invariância à Translação e Matrizes Circulantes I

Matrizes Circulantes

Uma matriz circulante é uma matriz Toeplitz especial onde cada linha é uma translação cíclica da linha anterior. Ela é completamente definida por sua **primeira coluna** (ou linha).

$$C = \text{circ}(k) = \begin{bmatrix} k_0 & k_{d-1} & \dots & k_1 \\ k_1 & k_0 & \dots & k_2 \\ \vdots & \vdots & \ddots & \vdots \\ k_{d-1} & k_{d-2} & \dots & k_0 \end{bmatrix}$$

Invariância à Translação e Matrizes Circulantes II

A Conexão Fundamental

A **convolução circular** de um sinal x com um kernel k é matematicamente **equivalente** à multiplicação do vetor x pela matriz circulante gerada por k :

$$k * x = \text{circ}(k)x$$

Operadores lineares que são invariantes à translação (equivariantes) são sempre matrizes circulantes.

Propriedades das Matrizes Circulantes I

Propriedades Algébricas e Computacionais

Matrizes circulantes possuem uma estrutura muito rica que as torna poderosas e eficientes:

- **Fechadas sob Operações Matriciais:** A soma, o produto e a inversa de matrizes circulantes são também matrizes circulantes.
- **Diagonalização pela DFT:** Todas as matrizes circulantes compartilham os mesmos autovetores, que são as bases da Transformada Discreta de Fourier (DFT). Isso significa que elas podem ser eficientemente diagonalizadas.

Expandindo para 2D: Convoluções em Imagens I

O Kernel como um Detector de Features

Em 2D, o kernel é uma pequena matriz de pesos que aprende a detectar um padrão espacial específico (uma borda, uma cor, uma textura). A operação é análoga à 1D: o kernel desliza sobre a imagem, calculando o produto escalar com a região correspondente.

Expandindo para 2D: Convoluções em Imagens II

Definição Matemática (Correlação Cruzada 2D)

A operação que chamamos de “convolução” em redes neurais é, tecnicamente, a **correlação cruzada**. Para uma imagem de entrada I e um kernel K , a saída (mapa de features) S em cada posição (i, j) é dada por:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$$

Esta é a soma dos produtos elemento a elemento entre o kernel e a sub-região da imagem sobre a qual ele está posicionado.

Expandindo para 2D: Convoluções em Imagens III

A Operação na Prática

Visualmente, a operação em uma única posição é um produto escalar entre as versões “achataadas” (flattened) da sub-região da imagem e do kernel:

$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \odot \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix} \Rightarrow \sum = a \cdot w_{11} + b \cdot w_{12} + \dots + i \cdot w_{33}$$

Reduzindo a Dimensão: A Camada de Pooling I

O que é Pooling?

A camada de pooling é uma operação de **downsampling** não-linear que resume uma vizinhança de neurônios em um único valor. A mais comum é a **Max-Pooling**.

- **Como funciona?** Uma janela (geralmente 2x2 com um stride de 2) desliza sobre o mapa de features, e apenas o valor **máximo** daquela janela é passado para a próxima camada.

Reduzindo a Dimensão: A Camada de Pooling II

Por que usar Pooling?

- 1 Redução Computacional:** Reduz drasticamente as dimensões espaciais, diminuindo o número de parâmetros e cálculos nas camadas seguintes.
- 2 Invariância Local:** Ao tomar o máximo de uma região, a representação se torna insensível a pequenas translações do feature dentro daquela janela. Se uma borda se move um pixel, a saída do pooling provavelmente permanecerá a mesma.

Reduzindo a Dimensão: A Camada de Pooling III

Mapa de Features (4x4)

1	2	4	1
8	3	2	5
2	4	3	9
1	1	1	2



Saída (2x2)

8	5
4	9

A Ideia Central da Convolução: Compartilhamento de Pesos

Revisitando o Problema do MLP

Em uma camada totalmente conectada (MLP), cada conexão tem um peso único. Isso leva a uma explosão no número de parâmetros e não aproveita a estrutura espacial das imagens.

A Ideia Central da Convolução: Compartilhamento de Pesos II

A Solução da CNN: Um Kernel para Todos

A camada convolucional implementa o **compartilhamento de pesos (weight sharing)**.

O **mesmo kernel** (a pequena matriz de pesos) é aplicado em todas as posições da imagem de entrada. Em vez de aprender um detector de bordas para o canto superior esquerdo e outro para o centro, a rede aprende **um único detector de bordas** e o reutiliza em toda a imagem.

A Ideia Central da Convolução: Compartilhamento de Pesos

III

As Consequências do Compartilhamento de Pesos

- 1 Eficiência de Parâmetros:** Em vez de milhões de pesos, precisamos aprender apenas os poucos pesos do kernel (e.g., 9 pesos para um kernel 3×3). Isso reduz drasticamente o risco de overfitting e a necessidade de dados.
- 2 Equivariância à Translação:** Esta é a consequência mais importante. Como o mesmo detector de features é usado em todos os lugares, se o objeto na imagem se desloca, o mapa de features resultante também se desloca, mas as ativações permanecem as mesmas. A capacidade da rede de detectar a feature se torna **independente da sua localização**.

Os Blocos de Construção de uma CNN I

A Camada Convolutiva

Consiste em múltiplos kernels. Cada kernel produz um **mapa de features** 2D. Se a camada tem 16 kernels, ela produzirá 16 mapas de features (canais).

A Função de Ativação (ReLU)

Após cada convolução, uma ativação não-linear (geralmente ReLU) é aplicada para permitir o aprendizado de features complexas.

Os Blocos de Construção de uma CNN II

A Camada de Pooling

A camada de pooling (e.g., Max-Pooling) realiza uma operação de **downsampling**.

- **O quê?** Reduz a dimensionalidade espacial dos mapas de features (e.g., pega o valor máximo em uma janela 2x2).
- **Por quê?** Torna a representação mais compacta e robusta a pequenas translações nos dados (invariância local).

Empilhando Camadas: Arquitetura LeNet-5 I

Aprendizado Hierárquico na Prática

Uma CNN típica é formada pelo empilhamento de blocos 'CONV -> RELU -> POOL', seguido por camadas totalmente conectadas ('Linear') no final.

Empilhando Camadas: Arquitetura LeNet-5 II

Arquitetura LeNet-5

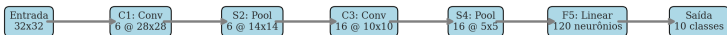


Figura: A arquitetura LeNet-5, uma das primeiras CNNs de sucesso. Observe a hierarquia: as primeiras camadas extraem features, e as últimas as utilizam para classificação.

Anatomia da Camada Convolutacional (Conv2D) I

Componentes Fundamentais

Uma camada convolutacional é definida por seus filtros (ou kernels) aprendíveis.

- **Canais de Entrada (C_{in}):** A profundidade do volume de entrada. Para a primeira camada em uma imagem RGB, $C_{in} = 3$. Para camadas internas, é o número de filtros da camada anterior.
- **Canais de Saída (C_{out}):** O número de filtros (kernels) na camada. Cada filtro aprende a detectar um padrão diferente. Este será o número de canais do volume de saída.
- **Tamanho do Kernel ($K \times K$):** As dimensões espaciais de cada filtro (e.g., 3×3 , 5×5).

Anatomia da Camada Convolutacional (Conv2D) II

Operação em Múltiplos Canais

Cada um dos C_{out} filtros é, na verdade, um volume de tamanho $(C_{in} \times K \times K)$. Ele realiza uma convolução 2D em cada canal de entrada separadamente e **soma** os resultados (mais um viés) para produzir um único canal de saída.

Hiperparâmetros da Convolução: Stride e Padding I

Stride (Passo)

O 'stride' controla o quanto o kernel se move a cada passo. Um stride de 1 move o filtro um pixel por vez. Um stride de 2 "pula" um pixel, resultando em um mapa de features menor.

Hiperparâmetros da Convolução: Stride e Padding II



Hiperparâmetros da Convolução: Stride e Padding III

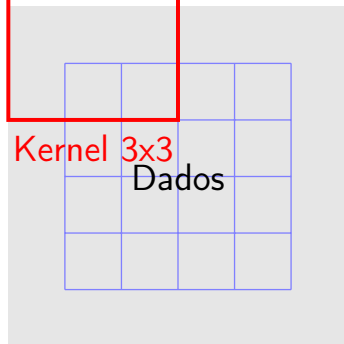
Padding (Preenchimento)

O 'padding' adiciona uma borda de zeros ao redor da imagem de entrada. É usado para:

- Controlar o tamanho espacial da saída. Com o padding correto, a saída pode ter o mesmo tamanho da entrada (convolução "same").
- Permitir que o kernel processe as bordas da imagem de forma mais completa.

Hiperparâmetros da Convolução: Stride e Padding IV

Entrada com Padding=1



Calculando as Dimensões da Saída I

A Fórmula da Convolução

Dado um volume de entrada de tamanho $(N, C_{in}, H_{in}, W_{in})$, uma camada convolucional com C_{out} filtros de tamanho $K \times K$, padding P e stride S , o volume de saída terá o tamanho $(N, C_{out}, H_{out}, W_{out})$, onde:

$$W_{out} = \frac{W_{in} - K + 2P}{S} + 1$$

$$H_{out} = \frac{H_{in} - K + 2P}{S} + 1$$

Calculando as Dimensões da Saída II

Exemplo:

Uma imagem de entrada de 32x32 com um kernel 5x5, padding 2 e stride 1 resulta em uma saída de 32x32.

$$W_{out} = \frac{32 - 5 + 2(2)}{1} + 1 = 32$$

Reduzindo a Dimensão: A Camada de Pooling I

O que é Pooling?

A camada de pooling é uma operação de **downsampling** não-linear que resume uma vizinhança. A mais comum é a **Max-Pooling**. Uma janela (geralmente 2x2 com stride 2) desliza sobre o mapa de features, e apenas o valor **máximo** da janela é passado para a próxima camada.

Reduzindo a Dimensão: A Camada de Pooling II

Por que usar Pooling?

- 1 Redução Computacional:** Reduz drasticamente as dimensões espaciais, diminuindo o número de parâmetros e cálculos nas camadas seguintes.
- 2 Invariância Local:** Ao tomar o máximo de uma região, a representação se torna insensível a pequenas translações da feature dentro daquela janela.

Mergulho Raso: O Backward Pass do Max-Pooling I

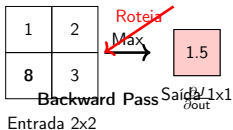
A Intuição: Roteando o Gradiente

O Max-Pooling não tem parâmetros para aprender, mas precisa propagar o gradiente para as camadas anteriores. A regra é simples: Durante o *forward pass*, para cada janela, lembramos a **posição** do elemento máximo.

Durante o *backward pass*, o gradiente que chega a uma saída do pooling é roteado **exclusivamente** para a posição de entrada que foi o máximo. Todos os outros gradientes de entrada naquela janela são zero.

Mergulho Raso: O Backward Pass do Max-Pooling II

Forward Pass



0	0
1.5	0

$$\frac{\partial J}{\partial \text{in}}$$

Figura: O gradiente de entrada '1.5' é passado apenas para a posição do '8', que foi o máximo no forward pass.

Indo Mais Fundo: Redes Residuais (ResNets) I

O Problema da Profundidade

Treinar redes muito profundas (e.g., > 20 camadas) é difícil. O gradiente pode desaparecer ou explodir durante o backpropagation, degradando a performance.

Indo Mais Fundo: Redes Residuais (ResNets) II

A Solução: Conexões Residuais (Skip Connections)

As ResNets introduziram uma “conexão de atalho” que pula uma ou mais camadas. A saída de um bloco se torna $\mathcal{F}(x) + x$.

- **Intuição:** É mais fácil para a rede aprender a função residual $\mathcal{F}(x)$ (a "diferença" que precisa ser adicionada) do que aprender a transformação completa a partir do zero.
- **Efeito:** Permite que os gradientes fluam mais facilmente através da rede, possibilitando o treinamento de modelos com centenas ou até milhares de camadas.

Invariância: Arquitetura vs. Aumento de Dados I

Invariância à Translação (Embutida na Arquitetura)

O compartilhamento de pesos na camada convolucional confere à CNN uma propriedade chamada **equivariância à translação**: se a entrada é deslocada, o mapa de features também é deslocado. A camada de pooling transforma essa equivariância em uma **invariância** local.

Invariância: Arquitetura vs. Aumento de Dados II

Outras Simetrias: Aumento de Dados (Data Augmentation)

E quanto a outras transformações, como **rotação**, **escala**, ou **espelhamento**? A CNN não é inerentemente invariante a elas. Nós "ensinamos" essas invariâncias ao modelo de forma explícita através do **aumento de dados**: durante o treinamento, apresentamos à rede versões aleatoriamente rotacionadas, redimensionadas e espelhadas das imagens. O modelo aprende que, apesar dessas transformações, o rótulo da classe permanece o mesmo.

Laboratório Prático (Colab) I

A Tarefa

No notebook do Colab que acompanha esta aula, vamos:

- 1 Construir uma CNN simples em PyTorch do zero.
- 2 Treiná-la para classificar imagens do dataset MNIST (ou Fashion-MNIST).
- 3 **Visualizar os filtros (kernels)** aprendidos pela primeira camada convolucional para entender que tipo de padrões simples a rede está detectando.

Laboratório Prático (Colab) II

Para a Próxima Aula

- **Tópico:** Arquiteturas de CNNs Modernas.
- **Objetivo:** Estudar as arquiteturas que venceram a competição ImageNet e se tornaram a base da visão computacional moderna (VGG, GoogLeNet, ResNet).

Referências I

-  Bach, Francis (2025). *Learning Theory from First Principles*.
URL: https://www.di.ens.fr/~fbach/ltpf_book.pdf.
-  Belkin, Mikhail et al. (2019). “Reconciling modern machine-learning practice and the classical bias–variance trade-off”. Em: *Proceedings of the National Academy of Sciences* 116.32, pp. 15849–15854.
-  Bishop, Christopher M. (2006). *Pattern Recognition and Machine Learning*. Springer.
-  Bishop, Christopher M. e Hugh Bishop (2023). *Deep Learning: Foundations and Concepts*. Springer.
-  Buchanan, Sam et al. (ago. de 2025). *Learning Deep Representations of Data Distributions*.
<https://ma-lab-berkeley.github.io/deep-representation-learning-book/>. Online.

Referências II

-  Cybenko, George (1989). “Approximation by superpositions of a sigmoidal function”. Em: *Mathematics of control, signals and systems* 2.4, pp. 303–314.
-  Domke, Justin (2011). *Automatic Differentiation and Neural Networks*. Lecture Notes, Statistical Machine Learning. URL: https://people.cs.umass.edu/~domke/courses/sml2011/08autodiff_nnets.pdf.
-  Gefter, Amanda (fev. de 2015). “The Man Who Tried to Redeem the World with Logic”. Em: *Nautilus*. URL: <https://nautil.us/the-man-who-tried-to-redeem-the-world-with-logic-235253/>.
-  Grünwald, Peter D. (2007). *The Minimum Description Length Principle*. The MIT Press.

Referências III

-  Hastie, Trevor et al. (2022). “Surprises in high-dimensional ridgeless least squares interpolation”. Em: *The Annals of Statistics* 50.2, pp. 949–986.
-  Hochreiter, Sepp e Jürgen Schmidhuber (1997). “Flat minima”. Em: *Neural computation* 9.1, pp. 1–42.
-  Jordan, Michael I (2019). “Artificial intelligence—the revolution hasn’t happened yet”. Em: *Harvard Data Science Review* 1.1.
-  Keskar, Nitish Shirish et al. (2017). “On large-batch training for deep learning: Generalization gap and sharp minima”. Em: *International Conference on Learning Representations (ICLR)*.
-  Kingma, Diederik P. e Jimmy Ba (2015). “Adam: A Method for Stochastic Optimization”. Em: *ICLR 2015*.
-  Krause, Andreas e Jonas Hübner (2025). *Probabilistic Artificial Intelligence*. arXiv: 2502.05244 [cs.AI].

Referências IV

-  Liu, Yuxi (jan. de 2024). *The Perceptron Controversy*. Website. URL: <https://yuxi.ml/essays/posts/perceptron-controversy/> (acesso em 10/09/2025).
-  Martins, André FT e Ramón F Astudillo (2016). “From softmax to sparsemax: A sparse model of attention and multi-label classification”. Em: *International conference on machine learning*. PMLR, pp. 1614–1623.
-  McCulloch, Warren S. e Walter Pitts (1943). “A Logical Calculus of the Ideas Immanent in Nervous Activity”. Em: *The Bulletin of Mathematical Biophysics* 5.4, pp. 115–133. DOI: 10.1007/bf02478259.

Referências V

-  Mercer, James (1909). “Functions of positive and negative type, and their connection with the theory of integral equations”. *Em: Philosophical transactions of the Royal Society of London. Series A* 209.441-458, pp. 415–446.
-  Minsky, Marvin e Seymour A Papert (1969). *Perceptrons: An introduction to computational geometry*. MIT press.
-  Montufar, Guido F et al. (2014). “On the number of linear regions of deep neural networks”. *Em: Advances in neural information processing systems* 27.
-  Murphy, Kevin Patrick (2023). *Probabilistic Machine Learning: Advanced Topics*. The MIT Press.
-  Nesterov, Yurii (2004). *Introductory lectures on convex optimization: A basic course*. Vol. 87. Springer Science & Business Media.

Referências VI

-  Oja, Erkki (1982). “A simplified neuron model as a principal component analyzer”. Em: *Journal of mathematical biology* 15.3, pp. 267–273.
-  Paes, Lucas Monteiro et al. (2023). “On the Inevitability of the Rashomon Effect”. Em: *IEEE Journal on Selected Areas in Information Theory* 4.2, pp. 416–427. DOI: 10.1109/JSAIT.2023.3287661.
-  Prince, Simon J.D. (2023). *Understanding Deep Learning*. The MIT Press.
-  Rasmussen, Carl Edward e Christopher K. I. Williams (2006). *Gaussian processes for machine learning*. The MIT Press.
-  Rosenblatt, Frank (1958). “The perceptron: a probabilistic model for information storage and organization in the brain.”. Em: *Psychological review* 65.6, p. 386.

Referências VII

-  Rudin, Cynthia et al. (2024). “Position: amazing things come from having many good models”. Em: *ICML*.
-  Schölkopf, Bernhard e Alexander J Smola (2002). *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press.
-  Shalev-Shwartz, Shai e Shai Ben-David (2014). *Understanding machine learning: From theory to algorithms*. Cambridge university press.
-  Shannon, C. E. (1988). “Programming a computer for playing chess”. Em: *Computer Chess Compendium*. Berlin, Heidelberg: Springer-Verlag, pp. 2–13. ISBN: 0387913319.
-  Shannon, Claude E. (1948). *A Mathematical Theory of Communication*. Vol. 27, pp. 379–423, 623–656.

Referências VIII

-  Turing, Alan M. (1969). "Intelligent Machinery". Em: *Machine Intelligence 5*. Ed. por Bernard Meltzer e Donald Michie. Edinburgh University Press, pp. 3–23.
-  Vapnik, Vladimir (1998). *Statistical learning theory*. Wiley.
-  Vieira, Tim (ago. de 2017). *Backprop is not just the chain rule*. Blog Post. URL: <https://timvieira.github.io/blog/post/2017/08/18/backprop-is-not-just-the-chain-rule/> (acesso em 15/09/2025).